Program Synthesis from Polymorphic Refinement Types

Varun Varma

September 22nd, 2020

Abstract

Program synthesis is the problem of generating programs that satisfy some specification. Typically, a program that is generated from a synthesis algorithm needs to be verified to fit the specification This paper asserts that scalable program synthesis requires modular verification of programs. Modular verification can approach solutions to each constraint independently.

Types are a form of constraints and can be used as the specification of a program synthesis problem. Alternatives like input out pairs cannot be verified as easily and provide weaker guarantees on the validity of a program. Normal types are not expressive enough to for program synthesis specification, so refinement types must be used. Refinement types are predicates embedded into the type.

The paper introduces an algorithm for program synthesis using refinement types. Type checking is done with a liquid type inference algorithm. Other type inference algorithms like Hindley-Milner inference and horn clause satisfiability cannot be used because they are bottom up and therefore can only type check a program after it is created. A top down system like liquid type inference is better suited for program synthesis because it can type check as the program space is being enumerated. An essential element of liquid type inference is decomposing specification to achieve modularity. Some expressions, like guards, can be easily decomposed but others can decompose infinitely. The paper presents the *MUSFix*, a bidirectional type checker that enables solving sub-constraints incrementally, to solve this issue.

The paper presents a prototype of the algorithms discussed named *SYNQUID*. The target program and specification were written in a ML-like terminating functional language featuring conditionals, algebraic data types, pattern matching, and polymorphism. A program synthesis is

problem is defined by its specification refinement type and its typing environment. *SYNQUID* works by constructing a candidate solution to a constraint or sub constraint, either inserting an expression or further decomposing the constraint, and backtracking if the type checking fails. It was tested on a benchmark set consisting of 64 synthesis problems pulled from various sources. *SYNQUID* was shown to be able to synthesize programs more complex than those previously reported and handle most of its competitors' most challenging benchmarks (Polikarpova, Kuraj, & Solar-Lezama, 2016).

Materials and Methods

A benchmark set of 64 program synthesis problems was constructed, each problem taken from a variety of sources. Each program synthesis problem was defined by its specification refinement type and its typing environment. A prototype of the program synthesis algorithm introduced in the paper, called SYNQUID, was created and tested on these benchmarks. No specific processer is specified for the benchmark. The results of each benchmark were recorded and are available at Table 1 (Polikarpova, Kuraj, & Solar-Lezama, 2016).

Analysis

Table 1 - Benchmarks and Synquid Results

Group	*Description*	*#G*	*Components*	*#M*	*Spec*	*Code*	*T-all*	*T-def*	*T-nrt*	*T-ncc*	*T-nmus*
List	is empty	1	true, false	1	6	6	0.02	0.02	0.02	0.02	0.01
	is member	1	true, false, $=$, $\neq$	2	6	18	0.11	0.11	0.13	0.10	-
	duplicate each element	1		1	7	16	0.05	0.05	-	0.08	0.04
	replicate	1	0, inc, dec, $\leq$, $\neq$	1	4	21	0.05	0.05	9.63	0.05	-
	append two lists	1		1	8	15	0.15	0.09	-	0.13	0.10
	concatenate list of lists	1	append	3	5	12	0.05	0.05	0.22	0.04	0.04
	take first n elements	1	0, inc, dec, $\leq$, $\neq$	1	8	27	0.12	0.12	55.82	0.12	-
	drop first n elements	1	0, inc, dec, $\leq$, $\neq$	1	11	20	0.10	0.10	7.87	0.09	-
	delete value	1	$=$, $\neq$	2	8	26	0.10	0.10	0.17	0.12	-
	map	1		1	5	22	0.03	0.03	0.06	0.03	0.02
	zip	1		1	10	22	0.08	0.08	-	0.10	0.07
	zip with function	1		1	10	33	0.07	0.07	-	0.17	0.06
	cartesian product	1	append, map	3	8	26	0.30	0.29	5.83	0.25	0.23
	i-th element	1	0, inc, dec, $\leq$, $\neq$	1	12	20	0.05	0.05	0.38	0.05	-
	index of element	1	0, inc, dec, $=$, $\neq$	2	8	20	0.08	0.08	0.14	0.07	-
	insert at end	1		2	21	19	0.10	0.10	0.24	0.11	0.12
	reverse	1	insert at end	2	15	12	0.09	0.10	0.29	0.12	0.09
	foldr	1		2	14	32	0.10	0.10	-	0.10	0.44
	length using fold	1	0, inc, dec	2	4	17	0.03	0.07	0.03	0.03	0.02
	append using fold	1		2	8	20	0.04	2.19	0.05	0.04	0.03
Unique list	insert	1	$=$, $\neq$	2	8	26	0.27	0.22	0.85	0.20	-
	delete	1	$=$, $\neq$	2	8	22	0.18	0.19	1.07	0.26	-
	remove duplicates	2	is member	2	13	47	0.36	0.87	0.72	0.33	-
	remove adjacent dupl.	1	$=$, $\neq$	3	5	32	1.33	1.32	-	1.31	-
	integer range	1	0, inc, dec, $\leq$, $\neq$	2	13	23	2.36	2.33	22.27	2.33	-
Strictly sorted list	insert	1	$<$	2	8	41	0.18	0.17	0.43	0.16	-
	delete	1	$<$	2	8	29	0.10	0.09	0.21	0.10	-
	intersect	1	$<$	2	8	40	0.33	0.32	0.68	0.34	-
Sorting	insert (sorted)	1	$\leq$, $\neq$	2	8	34	0.25	0.24	0.68	0.23	-
	insertion sort	1	insert (sorted)	4	5	12	0.06	0.06	0.20	0.06	0.05
	sort by folding	1	foldr, $\leq$, $\neq$	2	11	47	2.14	-	-	2.21	-
	extract minimum	1	$\leq$, $\neq$	4	23	40	4.28	4.35	-	7.58	-
	selection sort	1	extract minimum	6	5	16	0.49	0.44	-	0.42	0.38
	balanced split	1		4	31	33	0.96	0.51	-	1.40	0.80
	merge	1	$\leq$, $\neq$	2	17	41	2.19	14.61	-	6.85	-
	merge sort	1	split, merge	6	11	25	2.10	2.10	-	2.52	1.69
	partition	1	$\leq$	4	27	40	2.84	7.89	-	3.42	-
	append with pivot	1		2	28	22	0.22	0.15	0.58	0.22	0.19
	quick sort	1	partition, append w/pivot	6	11	22	2.71	18.45	-	2.49	4.94
Tree	is member	1	false, not, or, $=$	2	6	28	0.29	0.29	7.90	0.28	-
	node count	1	0, 1, +	1	4	18	0.20	0.20	-	0.91	0.14
	preorder	1	append	2	5	18	0.21	0.20	-	0.91	0.15
	create balanced	1	0, inc, dec, $\leq$, $\neq$	2	7	29	0.14	0.15	-	0.21	-
BST	is member	1	true, false, $\leq$, $\neq$	2	6	37	0.09	0.08	0.10	0.08	-
	insert	1	$\leq$, $\neq$	2	8	55	0.91	0.88	-	0.82	-
	delete	1	$\leq$, $\neq$	2	8	68	5.68	5.62	-	10.74	-
	BST sort	5	$\leq$, $\neq$	6	51	115	1.38	1.35	-	1.25	-
Binary Heap	is member	1	false, not, or, $\leq$, $\neq$	2	6	43	0.38	0.38	9.63	0.35	-
	insert	1	$\leq$, $\neq$	2	8	55	0.51	0.50	8.83	0.48	-
	1-element constructor	1	$\leq$, $\neq$	2	5	8	0.02	0.02	0.02	0.02	0.02
	2-element constructor	1	$\leq$, $\neq$	2	6	55	0.08	0.08	0.25	0.07	-
	3-element constructor	1	$\leq$, $\neq$	2	7	246	2.10	2.12	-	1.98	-
AVL	rotate left	3	inc	3	104	91	11.08	12.43	-	17.06	10.08
	rotate right	3	inc	3	107	91	19.23	18.34	-	36.35	17.87
	balance	1	rotate, nodeHeight, isSkewed, isLHeavy, isRHeavy	4	31	119	1.56	-	-	1.76	-
	insert	1	balance, $<$	3	22	47	1.84	1.81	-	1.64	-
	extract minimum	1	$<$	5	11	25	1.92	1.87	-	1.72	-
	delete	2	extract minimum, balance, $<$	5	37	63	15.67	-	-	13.79	-
RBT	balance left	2		9	143	137	5.62	5.53	-	48.47	-
	balance right	2		9	144	137	7.63	7.72	-	45.32	-
	insert	3	balance left, right, $\leq$, $\neq$	9	49	112	8.95	8.53	-	7.93	-
User	desugar AST	1	0, 1, 2	4	5	46	1.17	1.10	-	1.23	0.78
	make address book	1	is private	3	5	35	0.62	3.67	-	0.94	0.55
	merge address books	1	append	3	8	19	0.35	5.85	-	0.31	0.24

Table 1 shows the benchmark results. Each problem is organized in its category of problem and the number of defined measures, size of specification, and size of synthesized program (by counting abstract syntax tree nodes) are recorded. The SYNQUID synthesis times (in seconds) with minimal bounds, default bounds, without round-trip type checking, without the

type consistency check, and without MUSix are recorded as well. All benchmarks passed. This suggests that SYNQUID, and refinement type-based program synthesis engines like it, are effective and efficient tools of program synthesis.

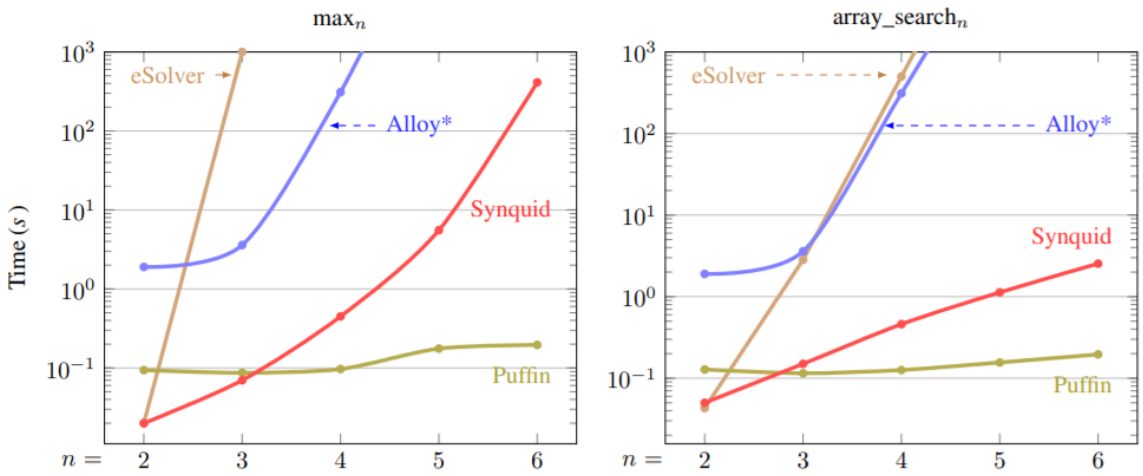

Figure 1 - Evaluation on non-recursive benchmarks

Figure 1 shows comparisons of four different programs that are each generated from different program synthesis engines, including SYNQUID, on two different program synthesis specifications that operate on lists. The length of the input array is represented on the X axis and time, in seconds, is represented on the Y axis. A lower time is better because it suggests the synthesis engine is more efficient. The graph shows that SYNQUID, despite its novel use of refinement types, is competitive with or superior to state-of-the-art program synthesis engines.

My research paper intends to introduce a program synthesis engine capable of efficiently reusing past solutions for a new specification. The main difficulty with this task is identifying where to reuse a past program, because the specifications are often not expressive enough. Refinement types are an effective way of creating a specification that has all the benefits of type checking but can also express complex specifications. In addition, refinement types can be deconstructed so a search algorithm can find similarity between the individual components of a deconstructed specification to find compatible past solutions more accurately. This paper not only introduces a program synthesis prototype that uses refinement types, but also demonstrates that it is an efficient method of program synthesis that can compete with the current state of the art synthesis engines.

Bibliography

Polikarpova, N., Kuraj, I., & Solar-Lezama, A. (2016). Program synthesis from polymorphic refinement types. *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2016*. doi:10.1145/2908080.2908093